

EXP.NO: 4

EDA-Data Inspection and Analysis

AIM:

- Viewing and inspecting DataFrames
- Filtering and subsetting data using conditions
- Descriptive statistics: measures of central tendency (mean, median, mode) and measures of dispersion (range, variance, standard deviation)

PROGRAM

```
import pandas as pd
```

```
data = {  
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male'],  
    'Married': ['Yes', 'No', 'Yes', 'Yes', 'No'],  
    'Dependents': ['0', '1', '3+', '2', '0'],  
    'Education': ['Graduate', 'Graduate', 'Not Graduate', 'Graduate', 'Graduate'],  
    'Self_Employed': ['No', 'Yes', 'No', 'No', 'Yes'],  
    'ApplicantIncome': [5720, 3076, 5000, 2340, 3276],  
    'CoapplicantIncome': [0, 1500, 2000, 2546, 0],  
    'LoanAmount': [110, 126, 208, 100, 78],  
    'Property_Area': ['Urban', 'Urban', 'Urban', 'Rural', 'Urban']  
}
```

```
df = pd.DataFrame(data)
```

```
print("First 5 Rows")
```

```
print(df.head())
```

```
print("\nLast 5 Rows")
```

```
print(df.tail())
```

```
print("\nData Information")
```

```
df.info()
```

```
print("\nDescriptive Statistics")
```

```
print(df.describe())
```

```

print("\nColumn Names")
print(df.columns)

categorical_cols = [
    'Gender',
    'Married',
    'Dependents',
    'Education',
    'Self_Employed',
    'Property_Area'
]

for col in categorical_cols:
    print(f"\nUnique values in {col}")
    print(df[col].unique())

df['Dependents'] = df['Dependents'].replace('3+', 3)

df['Dependents'] = pd.to_numeric(
    df['Dependents'],
    errors='coerce'
)

print("\nValue Counts of Dependents")
print(df['Dependents'].value_counts())

cols = [
    'ApplicantIncome',
    'CoapplicantIncome',
    'LoanAmount'
]

for col in cols:
    print(f"\nStatistics for {col}")
    print("Mean:", round(df[col].mean(), 2))
    print("Median:", round(df[col].median(), 2))
    print("Mode:", df[col].mode().values)
    print("Range:", round(df[col].max() - df[col].min(), 2))
    print("Variance:", round(df[col].var(), 2))
    print("Standard Deviation:", round(df[col].std(), 2))

```

OUTPUT:

```
First 5 Rows
  Gender Married Dependents Education Self_Employed ApplicantIncome \
0  Male    Yes           0   Graduate           No           5720
1  Female  No            1   Graduate           Yes           3076
2  Male    Yes           3+  Not Graduate           No           5000
3  Female  Yes           2   Graduate           No           2340
4  Male    No            0   Graduate           Yes           3276
```

```
CoapplicantIncome  LoanAmount Property_Area
0                0         110      Urban
1             1500         126      Urban
2             2000         208      Urban
3             2546         100      Rural
4                0          78      Urban
```

```
Last 5 Rows
  Gender Married Dependents Education Self_Employed ApplicantIncome \
0  Male    Yes           0   Graduate           No           5720
1  Female  No            1   Graduate           Yes           3076
2  Male    Yes           3+  Not Graduate           No           5000
3  Female  Yes           2   Graduate           No           2340
4  Male    No            0   Graduate           Yes           3276
```

```
CoapplicantIncome  LoanAmount Property_Area
0                0         110      Urban
1             1500         126      Urban
2             2000         208      Urban
3             2546         100      Rural
4                0          78      Urban
```

```
Data Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
```

```
Column Names
Index(['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed',
      'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Property_Area'],
      dtype='object')

Unique values in Gender
['Male' 'Female']

Unique values in Married
['Yes' 'No']

Unique values in Dependents
['0' '1' '3+' '2']

Unique values in Education
['Graduate' 'Not Graduate']

Unique values in Self_Employed
['No' 'Yes']

Unique values in Property_Area
['Urban' 'Rural']

Value Counts of Dependents
Dependents
0    2
1    1
3    1
2    1
Name: count, dtype: int64

Statistics for ApplicantIncome
Mean: 3882.4
Median: 3276.0
Mode: [2340 3076 3276 5000 5720]
Range: 3380
Variance: 2005700.8
Standard Deviation: 1416.23
```

```
Data columns (total 9 columns):
# Column Non-Null Count Dtype
0 Gender 5 non-null object
1 Married 5 non-null object
2 Dependents 5 non-null object
3 Education 5 non-null object
4 Self_Employed 5 non-null object
5 ApplicantIncome 5 non-null int64
6 CoapplicantIncome 5 non-null int64
7 LoanAmount 5 non-null int64
8 Property_Area 5 non-null object
dtypes: int64(3), object(6)
memory usage: 492.0+ bytes
```

```
Descriptive Statistics
ApplicantIncome CoapplicantIncome LoanAmount
count  5.000000  5.000000  5.000000
mean    3882.400000  1209.200000  124.400000
std     1416.227665  1164.183491  49.867825
min     2340.000000  0.000000  78.000000
25%     3076.000000  0.000000  100.000000
50%     3276.000000  1500.000000  110.000000
75%     5000.000000  2000.000000  126.000000
max     5720.000000  2546.000000  208.000000
```

```
Column Names
Index(['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed',
      'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Property_Area'],
      dtype='object')
```

```
Unique values in Gender
['Male' 'Female']
```

```
Unique values in Married
['Yes' 'No']
```

```
Unique values in Dependents
['0' '1' '3+' '2']
```

```
Unique values in Education
```

```
Value Counts of Dependents
Dependents
0    2
1    1
3    1
2    1
Name: count, dtype: int64

Statistics for ApplicantIncome
Mean: 3882.4
Median: 3276.0
Mode: [2340 3076 3276 5000 5720]
Range: 3380
Variance: 2005700.8
Standard Deviation: 1416.23

Statistics for CoapplicantIncome
Mean: 1209.2
Median: 1500.0
Mode: [0]
Range: 2546
Variance: 1355323.2
Standard Deviation: 1164.18

Statistics for LoanAmount
Mean: 124.4
Median: 110.0
Mode: [ 78 100 110 126 208]
Range: 130
Variance: 2486.8
Standard Deviation: 49.87
```

RESULT:

Thus, the given program was written and executed successfully.